Stopping the stop: a self-labeling gate for premature termination in autonomous coding agents

Juan Lescano

29 August 2026

Abstract

A coding agent granted standing permission still ends its turn early: it names the next step instead of taking it, offers a menu instead of choosing, or hands the user a command it could have run. We describe a gate on the agent's stop path that detects this in three lanes of increasing cost – an audit of the agent's own escape hatch, 298 pattern labeling functions, and a 9B local judge – and re-wakes the agent when the closing message shows the pattern. Building the gate was straightforward; evaluating it was not, because evaluation needs labels and labels need a reader. We show that two usable labels are already present in the transcript and cost no annotation. Before the gate exists, the developer's own next message says whether they had to ask for work the agent could have done. After the gate intervenes, the agent's tool calls before the developer speaks again say whether the intervention bought anything. Measured over 854 closings from 66 real sessions, the first label puts the base rate of premature stopping at 13.9% and the shipped gate's precision at 0.15 while it fires on 74% of closings: barely above chance, and we report that without softening it. The label then pays for itself twice. It shows that an exemption the design was confident in – releasing turns that wait on launched work – was releasing turns worse than average, and it selects a judge prompt that dominates the shipped one on precision, recall and intervention count at once. We report negative results in full, including three separate instructions a 9B judge would not hold, and two failed attempts to obtain a usable confidence signal from it.

1 Introduction

A coding agent is given standing permission to act: edit files, run builds, commit. It still stops early. It names the next step instead of taking it, offers a menu instead of picking, reports a result and asks whether to continue, or hands the user a command the agent could have run itself. The user's reply is then one word. That word costs a round trip, breaks the user's attention, and carries no information the agent did not already have.

The failure is not a capability failure. The same agent, woken with nothing more than "keep going," usually finishes the work. It is a stopping-policy failure, and it is invisible to the agent, which believes each turn ended reasonably.

This paper describes a gate that sits in the agent's stop path and re-wakes it when the closing message shows the pattern. That part is straightforward. The part that took the work, and the part we think transfers, is the second loop: how the gate discovers which of its own interventions were wrong, from signals already present in the transcript, with no human labeling.

We contribute:

1. A three-lane gate that spends a regular expression before it spends a model, and a model before it spends the user's attention (Section 3).

2. A weak-supervision label for premature stopping that requires no annotation: the user's own next message (Section 4).
3. An evaluation over 854 real closings from 66 sessions, including an ablation of each lane against that label (Section 5).
4. A self-labeling loop that grades each intervention by what the agent did afterwards, and names patterns for demotion on its own (Section 6).
5. Negative results, in full, including two failed attempts to obtain a usable confidence signal from the judge (Section 7).

2 Background and related work

Where the gate runs. Claude Code exposes a *Stop hook*: a program invoked when the agent finishes a turn. Exit status 0 lets the turn end. Exit status 2 discards the ending and wakes the agent with the hook's message on standard error. The hook sees the full transcript path, the working directory, and a flag marking whether it is already inside a re-wake chain. It is, in effect, an interposition point on the agent's stopping decision.

Model-as-judge. Using one language model to score another's output is now standard practice, and its failure modes are documented: position bias, verbosity bias, and self-preference [8]. Our judge is smaller than the agent it judges by roughly two orders of magnitude, which removes self-preference and forces the prompt to carry the whole task.

Self-critique and refinement. Self-Refine [4] and Reflexion [6] have a model improve its own output from its own feedback. Constitutional AI [1] supplies the critique from written principles. Our setting differs in one respect that matters: the agent has already decided it is done, so a self-critique would be asking the party that made the error to detect it. The gate is deliberately external.

Process supervision. Rewarding intermediate steps rather than final answers improves reliability [3]. Premature stopping is a process defect with a correct final answer – the work the agent did is fine – so it is invisible to outcome supervision.

Weak supervision. Snorkel [5] builds training labels from noisy programmatic labeling functions rather than annotators. Our pattern lane is a set of labeling functions in exactly that sense, and Section 6 adds the piece Snorkel assumes you have: an estimate of each function's accuracy, obtained here from the agent's later behaviour.

Calibration. Language models carry some signal about their own correctness [2], and self-consistency across samples is a usable proxy [7]. Section 7 reports that both failed here at 9B scale, and that token log-probabilities did not.

3 System

The gate is eight checkers and one local model behind a single orchestrator: 5389 lines of Python and 18 test files. Three lanes decide, cheapest first, and a lane that decides ends the turn's evaluation.

3.1 Lane 0: auditing the escape hatch

A closing message that begins with `BLOCKED:` is exempt from every other check. An agent that cannot proceed must be able to say so, and a gate that punishes a genuine blocker teaches the agent to lie about its state.

That exemption is also the cheapest escape in the system, and it was used as one. Lane 0 therefore audits the claim against the transcript rather than the prose. Three signals fail the audit: no tool call at all in the turn that declared the blocker, so nothing was tried; four or more distinct blockers named at once, which is a survey rather than a wall; and three consecutive blocked closings, which is a loop. A blocker that survives the audit passes untouched.

An agent that survives the audit and still needs out is given a release phrase drawn from a cryptographic random source, generated per block and present in no file the agent can read. Quoting it back proves the agent read the current block, and the blocker it accompanies is then audited against what actually ran that turn. The phrase exists because every fixed escape token we tried was eventually emitted from memory rather than from the message.

3.2 Lane 1: patterns as labeling functions

The second lane is 298 regular expressions over the agent's own words, grouped into nine classes by what the closing is doing rather than by what it says: extttask, extttannounce, extttdefer, extttwait, exttthandoff, exttterrand, extttread, extttidle and extttack. Appendix efapp:classes gives the full table. The grouping matters because the loop of Section efsec:loop tunes classes rather than individual expressions: one regular expression rarely fires often enough to be judged on its own.

Two design points matter more than the patterns themselves.

Quoted text does not count. Fenced code blocks, markdown tables, inline backticks, quotation marks and reported speech are stripped before matching. Without this, an agent that describes the gate trips it, and a report containing a table of results is condemned by its own table. This was not hypothetical: it fired on this project's own status reports twice before the table rule was added.

Patterns carry a confidence. A class written `ask:` decides on its own. A class written `ask?:` raises a doubt and defers to the model. Of 298 patterns, 275 are firm and 23 are doubts. This split is what makes a local model affordable: most closings never reach it, and the ones that do are the ones a regular expression was going to get wrong.

3.3 Lane 2: a small local judge

The third lane is a 9B parameter model served locally through Ollama, pinned by version, answering with exactly one token: `STOP` or `OK`. It runs at temperature 0 with a fixed seed.

The judge exists for one distinction a regular expression cannot make. In this corpus the Spanish word *falta* appears both as "work is missing" and as "it was missing and I fixed it." No amount of pattern engineering separates those; reading the sentence does.

Two implementation details proved necessary rather than cosmetic. First, all judge requests are serialized behind a machine-wide lock: the serving daemon batches concurrent requests, and batching changes the answer, so two test suites running at once produced different verdicts on identical input. Second, when the judge is asked to quote the offending sentence back to the agent, the prompt is strictly extractive and forbids translation; without that, the model paraphrased the sentence into English and the quote failed validation.

3.4 The message the agent receives

A block writes one short message to the agent. Its wording went through several revisions and the final form is deliberate: it states that the work already done stands, that permission was granted in advance and does not expire, and that if anything remains the agent should take the step rather than name it.

The reasoning is mechanical rather than courteous. An agent that reads an accusation spends the woken turn defending itself, which is precisely the outcome the block was meant to prevent. We observed this directly with earlier, blunter wording.

The same reasoning keeps the diagnostics out of the message. Which patterns fired, how confident the judge was, whether the lanes disagreed – all of it is useful for tuning and all of it reads as prosecution evidence. It is written to a private log instead. The agent receives the request; the maintainer receives the numbers. Section 6 is built on that log.

4 Experimental setup

4.1 Corpus

We use every session transcript on one developer's machine: 66 transcripts spanning 2026-08-03 to 2026-08-29, across several projects. From these we extract *closing pairs*: the agent's final text message of a turn together with the user's next message. This yields 854 pairs.

The corpus is naturally split by a date. The gate was first installed on 2026-08-28, so 55 transcripts are *ungated* – produced with no gate in the stop path – and 11 are *gated*. Ungated transcripts are the ones the gate never influenced, and all evaluation against the outcome label is run on them.

4.2 A label nobody had to write

Evaluating a stop gate requires knowing which stops were premature. Annotating that by hand is expensive and, worse, is annotated by the same person whose intuitions built the patterns.

The corpus already contains the judgement. When an agent stops early, the user's next message is a bare push: *dale*, *seguí*, *continue*, *keep going*. It carries no new instruction, because none was needed. We detect it with a closed lexicon anchored to the whole message, so that "continue with the refactor" is not counted and "ok dale" is.

This label is noisy in one direction and clean in the other. A push is strong evidence the turn ended too early. Its absence is weak evidence of the opposite, because a user often absorbs the loss and just answers the question. We therefore report recall against pushes and firing rate, and do not report precision, which this label cannot support.

4.3 Metrics

For each lane configuration we report how many closings it fires on, and what fraction of pushed closings it catches. We deliberately avoid a single combined score: the two costs are not commensurable. A missed premature stop costs the user one round trip. A false block costs the agent a full turn and, if the wording is wrong, poisons the next one.

5 Results

5.1 The corpus

The 66 transcripts yield 854 closing pairs. Closings are long: the median is 565 characters and the 90th percentile is 2866, which is already a warning about pattern matching, since a longer message gives a fixed pattern set more chances to fire on prose that is not a request.

Thirteen closings (1.5%) are harness error pages rather than messages the agent chose to write: an API error or a usage-limit notice left as the last assistant text. These are excluded from every lane. They matter more than their share suggests: 4 of the 11 user pushes in the whole corpus follow one. The user was pushing past an infrastructure failure, and a gate that reads those as premature stopping is blocking a session that has already ended.

5.2 What each lane fires on

Lane configuration	Fires on	Share
Firm patterns only	304	36.1%
Any pattern (firm + doubt)	539	64.1%
Judge only	569	67.7%
Firm patterns or judge (shipped)	620	73.7%
Firm patterns and judge	253	30.1%

Table 1: Firing rates over the 841 non-noise closings.

The shipped configuration fires on nearly three closings in four. That number is the most uncomfortable result here and we report it without softening. It has two readings and we cannot separate them with this data. Either the agent genuinely ends most turns with something nameable left open – which matches the authors' experience and is why the gate was built – or the gate is far too eager and the tolerable rate is much lower.

What we can say is that the two lanes are not redundant. Firm patterns and the judge agree on only 253 closings, while their union is 620. Each catches a large set the other misses, which is the intended division of labour: patterns catch fixed phrasings cheaply, the judge catches the rest.

5.3 Cost

The judge answers in 0.31 seconds at the median and 0.66 at the 90th percentile, running locally on consumer hardware alongside the agent's own work. The per-pattern confidence split is what keeps this affordable: 235 closings reach the judge as doubts rather than being decided by a regular expression, and the 304 firm hits never pay for a model call at all.

5.4 Against the push label

We evaluate on the 664 ungated closings, of which 6 non-noise closings were followed by a bare user push.

The shipped configuration catches every push. So does the judge alone, and so does the pattern lane alone once doubts are included. Firm patterns alone catch half.

This is a weak result and we want to be exact about why. Six positives cannot distinguish between configurations that all fire on more than a third of the corpus; a detector that fired on everything would also score 6 of 6. The label establishes that no lane is systematically

Lane configuration	Fires on	Pushes caught
Firm patterns only	238	3 of 6
Any pattern	426	6 of 6
Judge only	478	6 of 6
Firm or judge (shipped)	514	6 of 6
Firm and judge	202	3 of 6

Table 2: Recall against the user-push label, ungated closings only.

blind to the failure it targets, and nothing more. Precision is the quantity that matters here and this label cannot measure it, which is the entire reason for the loop in Section 6.

5.5 A stronger label, and what it costs to trust it

Bare pushes are too rare to tune against. But the lexicon was only ever an approximation of the question we care about, which is whether the developer's reply asked for work the agent could have done on its own.

We therefore ask that question directly, with a second local judge that reads the developer's reply rather than the agent's closing. It is given the closing for context and asked a single question: did the developer have to ask for work the agent could have done in the turn that just ended? A reply that pushes, picks from a menu the agent offered, or tells the agent to run what it just described is a push. A reply that supplies information the agent could not have had – a preference, a priority, a fact from outside the repository, a correction – is not.

This judgement is deliberately not the gate's own. The gate's judge reads the agent's closing; this one reads the reply. Grading a gate with the prompt it decides by measures its consistency and nothing else.

Two checks on the label before using it. It recovers 10 of the 11 bare pushes the lexicon found, without being given the lexicon. And on a held-out set of nine hand-written boundary cases – including short, blunt replies that carry real information, which are the ones a length heuristic gets wrong – it scores 9 of 9.

It labels 117 of the 841 non-noise closings as pushes: a base rate of **13.9%**.

5.6 The gate fires far more often than the failure occurs

That base rate is the result that reframes everything above.

Lane configuration	Fires on	Precision	Recall	F1
Base rate (the failure itself)	117	0.139	1.00	—
Firm patterns only	304	0.19	0.50	0.28
Any pattern	539	0.16	0.72	0.26
Judge only	569	0.15	0.75	0.26
Firm or judge (shipped)	624	0.15	0.81	0.26
Firm and judge	249	0.21	0.44	0.28

Table 3: Every configuration against the push label. Precision at or near the base rate means the configuration carries almost no information about which turns ended early.

The shipped configuration fires on 74% of closings and is right 15% of the time, against a base rate of 13.9%. It is barely above chance. Requiring both lanes to agree raises precision to 0.21 and halves recall, which is the only configuration that clearly beats the base rate, and it still fires on 30% of all closings.

Per class, precision ranges from 0.30 (`idle`) down to 0.07 (`wait`) and 0.09 (`errand`) – two classes that fire *below* the base rate and are therefore worse than not firing. But demoting classes barely moves the shipped number, from 0.152 to 0.159 under a greedy search over all nine, because the judge fires on almost everything the patterns do and more.

Nor does the judge's confidence rescue it. Sliced into four bands, precision is 0.44 ($n = 9$), 0.16, 0.10 and 0.16 with increasing confidence: not monotonic, and flat everywhere the volume is.

We want to be careful about what this does and does not show. The label is one-sided: a developer often absorbs a premature stop and simply answers, which counts as NEW and caps measurable precision below its true value. So 0.15 is a floor rather than an estimate. What survives that caveat is the comparison of rates. The failure occurs on roughly one closing in seven and the gate intervenes on three in four. Whatever the true precision is, the gate is firing several times more often than the thing it is looking for happens.

5.7 Tuning the judge against the label

With a label in hand the judge's instruction becomes testable rather than a matter of taste. Its final line read *when unsure, answer STOP*, which is a plausible default for a gate whose false negatives cost a round trip. It is also, on this evidence, the reason the judge fired on two closings in three.

We ran three system prompts over all 841 labelled closings, changing only the closing instruction.

Prompt	Final instruction	Judge fires	Prec.	Rec.	F1
base	when unsure, answer STOP	569	0.155	0.812	0.256
loose	when unsure, answer OK	538	0.156	0.795	0.258
strict	a test, then OK when unsure	536	0.164	0.821	0.266

Table 4: Judge prompt ablation over 841 labelled closings.

Flipping the default on its own does almost nothing. The judge fires 31 times less often out of 841 and precision moves by 0.001. This is the third instruction in this work that a 9B model would not hold, after the background-work exception and the request for a confidence rating, and it is the cleanest demonstration: the sentence was changed to its opposite and the behaviour barely moved.

What moved the number was replacing the default with a test the model can apply. The *strict* prompt asks whether there is a concrete action the agent could take right now, in this repository, without information only the developer has; if what remains needs a preference or a priority, that is OK. It dominates the baseline on every axis at once – fewer interventions, higher precision, higher recall – so there is no operating point to choose between them, and it is what the gate now ships.

The effect is small in absolute terms. Precision moves from 0.152 to 0.159 against a base rate of 0.139, which closes about a seventh of the gap to the best configuration we measured. We report it because the direction is informative even where the magnitude is not: a small model given a criterion behaves differently from the same model given a disposition, and the disposition is what prompts of this kind are usually written in.

5.8 Removing an exemption

The same label settled a design question the system had answered by intuition. Turns waiting on work the agent had launched were exempt: work in flight, no pattern raised, turn released without consulting the judge at all. The reasoning was that the harness wakes the session when the work finishes, so waiting is the mechanism rather than a stall.

Over the transcripts, 56 turns took that exemption and 12 of them ended with the developer pushing: 21% against a base rate of 13.9%. The exemption was releasing turns *worse* than average, and it was the single most confidently held assumption in the design.

It is gone. Waiting no longer excuses a turn; it selects a different request, which points out that the launched work reports back on its own and asks for the next piece in the meantime.

5.9 Before and after the gate

The corpus splits at 2026-08-28, when the gate was installed. Pushes fall from 10 of 664 ungated closings (1.5%) to 1 of 190 gated ones (0.5%).

The direction is what the gate is for, and the result is not significant: a two-sided Fisher exact test gives $p = 0.47$. With a base rate near one percent, detecting a halving at conventional power would need several thousand closings per arm. We report it because the effect size is the one worth powering a future study on, not because this study establishes it.

5.10 The judge's confidence

Taking the probability the judge assigns to its own verdict token, the distributions overlap heavily: median 0.95 for STOP ($n = 569$) and 0.87 for OK ($n = 272$). 115 STOP verdicts and 92 OK verdicts fall below 0.75.

At the extremes the signal is real. On hand-checked cases it reads 0.26 to 0.53 on messages that are finished reports and 0.97 to 1.00 on genuine premature stops. But on the 30 closings that reach the judge carrying doubts alone, it stops 24, and among the 6 stopped below 0.75 exactly three are finished reports and three are real cuts. A threshold there buys one correct release and pays for it with one lost catch.

The confidence is therefore logged and used by nobody. It is informative to a maintainer reading a specific case and not safe to act on automatically, and we think the distinction is worth naming: a signal can be genuine, reproducible, and still not be a decision rule.

6 The self-labeling loop

Every decision the gate makes is appended to a private log: the lane that decided, the patterns that fired, the judge's verdict and confidence, the latency, and the first line of the message. The log records what was decided. It does not record whether the decision was right, and for the first weeks tuning meant reading transcripts by hand.

The missing label was already in the transcript, a few lines below the block. A block wakes the agent, and from there exactly one of two things happens: the agent goes and does work, or it writes another paragraph and stops again.

> **Grading rule.** The number of tool calls between a block and the next time the user speaks is the label for that block. Above a threshold, the block bought work. At or below it, the block bought a paragraph.

Nobody supplies this. It is read from the same transcripts the gate already opens. The report joins each logged decision back to its transcript, counts the intervening tool calls, and aggregates by pattern class and by judge confidence. When a class accumulates enough blocks and most of them bought nothing, the report names that class for demotion from firm to doubt.

The report runs at session start and stays silent unless a class is ripe, speaking at most once per day. A report that speaks on every session is a report that gets skipped, and the loop's output is consumed by an agent with the same attention economics as its user.

The first two blocks the loop graded had bought zero tool calls each, on the same pattern class that had been demoted by hand that morning after measuring 888 closings. The loop reproduced a hand measurement in one second.

7 Negative results

These cost more than the positive ones and generalize better.

The judge cannot report its own confidence. Asked to append a confidence rating to its verdict, the 9B model answered with the maximum every time, across every message we tried. The number was constant and therefore carried nothing.

Self-consistency did not separate either. Re-sampling the same message five times at temperature 0.8 produced five identical verdicts, including on messages the verdict got wrong. At this scale the model is confidently wrong in a stable way, so disagreement-based confidence [7] has nothing to measure.

Token log-probabilities did move. Taking the probability the model assigned to the single verdict token it emitted produced a number that varies with the message, and that separates at the extremes. It is the only one of the three that worked.

A 9B judge cannot hold an exception. Nineteen percent of the judge's blocks were turns in which the agent had correctly launched background work and was waiting on it. We tried to teach this in the prompt, first as prose and then as few-shot examples. Both failed: the model could not hold the exception against its own standing instruction to answer STOP when unsure, and the examples degraded unrelated verdicts. The fix was to remove the question. Waiting is now established deterministically from the transcript, before the judge is called at all. *An instruction a small model cannot hold is a decision that belongs outside the model.*

Nor a disposition. The judge's instruction ended with *when unsure, answer STOP*. Replacing it with its exact opposite, *when unsure, answer OK*, changed 31 verdicts out of 841 and moved precision by 0.001. Replacing it instead with a test the model can apply – is there an action available right now that needs nothing only the developer knows – moved precision, recall and intervention count together. Three instructions in this work went unheld by the same model, and the pattern across them is consistent: it follows a criterion it can evaluate against the text in front of it, and ignores a disposition about how to feel when unsure.

A shared accelerator breaks reproducibility. Two test suites run concurrently produced different verdicts on byte-identical input. The cause is request batching in the serving daemon. We first misdiagnosed this as model flakiness and then as a prompt defect, and "fixed" it by adding an example, which made it worse. Determinism required a fixed seed and a machine-wide lock.

An A/B harness must pin everything the code reads. Our first before/after comparison reported three regressions. Both arms had been given different working directories, so one checker saw files on one side only. The regressions vanished once the directory was pinned.

8 Threats to validity

One user, one idiom. The corpus comes from a single developer. The patterns carry that developer's phrasing, and the judge required an explicit reading list for River Plate Spanish because *quedó cableado* ("it is wired end to end") was being read as work in progress. Nothing here has been tested on a second user's transcripts.

The gate contaminates its own corpus. Gated transcripts were produced with the gate running, so their closings are partly the gate's own output distribution. This is why all label-based evaluation is restricted to ungated transcripts, and why the era comparison in Section 5 is observational rather than controlled.

The push label is one-sided. Absence of a push is not evidence the turn was complete. We report recall and firing rate for this reason and make no precision claim from this label.

The grading rule rewards motion. Counting tool calls after a block cannot see whether the work was any good. An agent that learned to emit cheap tool calls after every block would defeat it. We have not observed this, and it is the most obvious way the loop could be gamed.

Thresholds are tuned, not derived. The demotion threshold and the work threshold were chosen for the volume this log receives.

9 Discussion

Three things generalize beyond this system.

Put the decision where the capability is. The clearest single result is the background-work case: an exception that a 9B model could not hold in its prompt became trivial once it was decided by ten lines of transcript inspection. The temptation with a model in the loop is to route every judgement through it. The cheaper lanes should take everything they can prove.

Confidence is a router before it is a decision. Per-pattern confidence is what makes the local model affordable, because it decides which messages are worth a model at all. The judge's own confidence, by contrast, turned out to be publishable and not actionable. A signal can be real, be logged, be informative to a maintainer, and still not be safe to threshold.

Interventions leave labels. Any system that interrupts an agent gets a natural experiment for free: the agent's behaviour after the interruption. We suspect this generalizes to most guardrails. The evaluation problem for interventions is often not a missing label but an unread one.

10 Limitations and future work

The loop is self-correcting in one direction only. It finds patterns that block too much, because every block leaves a graded trace. Patterns that block too little leave nothing, because a turn that was allowed to end is a turn nobody looked at. Closing that side requires an outcome signal from allowed turns; the push label is the obvious candidate and is too sparse under the gate to serve.

Three extensions follow directly. Demotion is currently proposed and applied by an agent reading the report; promotion in the other direction is not implemented. The judge is a general instruction-tuned model, and the graded log is exactly the dataset needed to fine-tune a specialist. And no part of this has been run against a second user, which is the first thing that should happen.

11 Conclusion

A gate that stops an agent from stopping early is easy to build and hard to evaluate, because evaluation needs labels and labels need a reader. Both labels this system needs were already lying in the transcript: the developer's own reply before the gate exists, and the agent's own behaviour after the gate intervenes. Neither costs an annotation.

Reading them was uncomfortable and useful in the same measure. The gate fires on three closings in four and is right about one in seven, which is close to the rate at which the failure occurs at all. The first thing the labels overturned was the design's most confident exemption, which turned out to be releasing turns worse than average. The second was a prompt written as a disposition, replaced by one written as a test, which improved every number at once.

Neither correction came from a new idea. Both came from finally measuring something that had been sitting in the transcript the whole time.

A Pattern classes

The 298 patterns are grouped into nine classes by what the closing message is doing. A class name written with a trailing question mark marks a doubt, which defers to the judge instead of deciding.

Two classes need an exception that is decided outside the patterns. `ask` patterns are suppressed on long messages, because a long report that ends in a courtesy question is a report, not a request for permission. `wait` patterns are suppressed when the transcript shows the agent is waiting on work it launched itself.

B The judge's instruction

The judge's system prompt is a decision rule, not a persona. Its shape, in order: the agent has standing permission and never needs to ask; a definition of STOP listing the observable behaviours; a definition of OK; a paragraph on the most common hard case, in which the message reports real finished work *and* names something still open, which is STOP; a counterweight stating that a closed caveat is not a pending item, because an agent that learns to delete its warnings to get past the gate is worse than one that stops; a note that deferred work often dresses as a disclosure; a note that a conditional offer is a stop; a short reading list for River Plate Spanish; and a test in place of a default: is there a concrete action the agent could take right now without information only the developer has, with OK when unsure. That last line was measured against its two alternatives in Table 4.

Class	Patterns	Reads as	Example trigger
`announce`	69+15	a step named instead of taken	“next I will”, *paso a*
`ask`	56+1	a question it may answer itself	“do you want me to”, *querés que*
`defer`	32+2	work made conditional on being asked	“if you want, I can”
`wait`	31+2	deferring to the user’s clock	“I’ll wait”, “let me know when”
`handoff`	24	giving the work back	“over to you”, “your call”
`errand`	23	sending the user to run something	“run this and tell me”
`read`	21+2	asking what the repo already answers	“where is the spec”
`idle`	12+1	reporting that nothing happened	“still running”, “no changes”
`ack`	7	a bare acknowledgement	“done”, “ok”

Table 5: The nine pattern classes. The second column counts firm patterns plus doubts, where a doubt raises the message to the judge instead of deciding. 275 of the 298 are firm.

The counterweight paragraph was added after the gate began punishing useful warnings. It is the only part of the prompt that argues against the gate’s own purpose, and it is load-bearing.

C The grading rule

```
for each logged block:
    find the blocked message in its transcript
    count tool calls until the user speaks again
    (a tool result is not the user speaking)
    fewer than 3 tool calls ⇒ the block bought nothing

for each pattern class:
    if it has 8 or more graded blocks
    and 70% or more bought nothing:
        name it for demotion
```

The threshold of three tool calls exists because a woken agent commonly reads one or two files to re-establish context before deciding to stop again. Zero is too strict and would count that re-orientation as work.

References

[1] Y. Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*, 2022.

[2] S. Kadavath et al. Language Models (Mostly) Know What They Know. *arXiv:2207.05221*, 2022.

[3] H. Lightman et al. Let’s Verify Step by Step. *arXiv:2305.20050*, 2023.

[4] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS*, 2023.

[5] A. Ratner et al. Snorkel: Rapid Training Data Creation with Weak Supervision. *VLDB*, 2017.

[6] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*, 2023.

[7] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.

[8] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.